

Does Code Generation Cure Serialization Fragility?

An Invariance Audit of LLM Graph Reasoners

Machine Learning with Graphs (Spring 2026) • Final Project Proposal

Yehonatan Barel (yonibarel2@gmail.com)

Ram Kedem (ramkedem@mail.tau.ac.il)

Dolev Abudi (dolevabudi@mail.tau.ac.il)

Ido Azoulay (idoazoulay1@mail.tau.ac.il)

TL;DR. We test whether code generation, the strongest known LLM–graph interaction mode [2], is *invariant* to the graph-serialization changes that break direct prompting [3], or whether the fragility merely *relocates* into the code-writing step. A transcription-free control (Graph-as-Code) localizes exactly where it lives.

1 Background & Motivation

Setting. An LLM cannot consume a graph $G = (V, E)$ directly: every graph task requires a *serialization* $g : G \mapsto \text{text}$ [1]. A single graph admits infinitely many equivalent serializations (arbitrary node labels, arbitrary edge order, edge list vs. adjacency list, plain text vs. JSON vs. natural language). Graph neural networks are permutation-invariant by construction, $f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^T) = f(\mathbf{X}, \mathbf{A})$, a core inductive bias of this course. An LLM operating on token sequences carries no such guarantee.

What is known. (i) *Direct prompting is fragile.* Fatemi et al. [1] showed the encoding choice alone shifts accuracy by 4.8–61.8% on basic graph tasks (GraphQA). Herbst et al. [3] confirmed this systematically: across $>1.5\text{M}$ inferences on the Erdős suite [5], decomposing serializations into *node labeling*, *edge encoding*, and *syntax*, they found reindexing, edge reordering, and format changes alter outputs on the *same* graph. Fine-tuning reduces sensitivity to relabeling but can increase it to structure and format changes. The mechanism: serialization dictates the model’s *computational path* (a sorted adjacency list allows one linear pass; a shuffled edge list forces repeated scans). (ii) *Code is the strongest mode.* Finkelshtein et al. [2] compared prompting, tool-use, and code generation across 14 datasets and models from Llama to GPT-5, finding code generation strongest overall; on a synthetic shortest-path task it achieved zero error at all path lengths while other modes collapsed beyond 2 hops.

The gap. These lines do not compose, and the design space has one untested cell:

	Direct answering	Executed code
Accuracy	measured (weaker)	measured (strongest)
Invariance to serialization	measured (<i>fragile</i>)	never measured

Herbst et al. [3] never execute code: NetworkX/PyG appear only as *input* syntaxes. Finkelshtein et al. [2] measure average accuracy under a fixed serialization; their ablations perturb the *graph itself*, never its serialization. The closest prior work, CodeGraph [4], executes LLM-written programs on GraphQA and observes a smaller accuracy spread across encoders under code than under CoT. Suggestive, but not an invariance audit, for three reasons: aggregate spread is not instance-level consistency (per-graph errors cancel across encoders); GraphQA’s encoders entangle relabeling, phrasing, and syntax, and never vary *edge order*; and no failure decomposition is attempted. Their own results already hint at relocation: with the incident encoding, code-mode edge counting drops to 77% (vs. 98.8% for adjacency), consistent with an undiagnosed *transcription* failure.

Why the answer is open. *Cure hypothesis:* once the model emits `nx.shortest_path(G,a,b)`, the executed algorithm is invariant by definition. *Relocation hypothesis:* before code runs, the model must (a) *transcribe* the serialized edges into the program, a copying task whose error rate may depend on format, order, and length, and (b) *select* an algorithm, a decision known to be highly sensitive to semantically equivalent rephrasing [6, 7]. A design insight sharpens the question: Graph-as-Code [2] hosts the graph *in the environment* (a typed dataframe the model queries), involving no transcription at all, while one-shot synthesis from serialized text involves full transcription. Contrasting the two isolates precisely where fragility lives.

Why it matters. Reliability, not average accuracy, gates deployment: a system whose answer flips under an arbitrary relabeling of the same input is unsafe. We supply the missing robustness dimension for the field’s current best-practice recommendation.

2 Proposed Approach

Core idea. A factorial **invariance audit**: (perturbation axis) $\times$ (interaction mode). For each graph we generate K provably equivalent serializations, varying one axis at a time following Herbst et al. [3]: node relabeling, edge reordering, data structure, and syntax. Each variant is queried under four modes:

M1. Direct — single-turn answer from the serialized graph.

M2. CoT — chain-of-thought prompting.

M3. Code-1shot — the model emits a full Python program from the serialized text; we execute it in a sandbox.

M4. Graph-as-Code — the graph lives in a dataframe environment [2]; the model queries it iteratively, so no transcription occurs.

We measure **accuracy** and **consistency** (agreement across the K equivalent variants); wrong-but-stable behavior is itself informative.

Hypotheses.

H1 (cure).

Code modes are near-invariant while Direct/CoT are not.

H2 (relocation).

Code-1shot shows comparable variance, concentrated in transcription/algorithm choice, while Graph-as-Code does not, localizing fragility in the transcription step.

H3 (task/length dependence).

Invariance degrades as serialization length or shuffling grows.

Technical outline. (1) Sample graphs/tasks (§3); (2) generate variants; (3) run all modes at temperature 0 with fixed prompts; (4) execute generated code in an isolated sandbox (subprocess, resource/time limits); (5) score against NetworkX ground truth; (6) automatically classify every code failure: *transcription error* (extract the graph object built by the program; compare its normalized edge set to ground truth), *algorithmic error* (correct graph, wrong procedure), or *execution error* (exception/timeout). This taxonomy localizes the fragility and is a contribution in itself.

Novelty. First *instance-level* invariance audit of executed-code graph reasoning under axis-isolated perturbations (beyond the aggregate per-encoder accuracies of [4]); first decomposition of code failures into transcription vs. algorithm choice; first transcription-free control isolating where fragility lives; a direct, falsifiable stress test of the “use code” recommendation.

Stretch goal (explicitly optional). Text-attributed graphs, where we conjecture code parses structure but discards node-text semantics. Built-in pilot: on *disconnected nodes*, direct prompting scores $\leq 0.5\%$ [1] while a one-line program is exact.

3 Experimental Plan

Datasets. GraphQA [1] and a subset of the Erdős suite [5]: 5–6 tasks spanning difficulty (edge existence, node degree, connected/disconnected nodes, cycle check, shortest path, triangle counting). Both public; GraphQA graphs are small (5–20 nodes), keeping prompts and costs low.

Models. Primary: one API reasoning-tier model of the o4-mini class, matching Finkelshtein et al. [2] so their published numbers (and the direct-mode numbers of [1, 3]) serve as pipeline sanity checks. Their scale study shows even small models benefit from code modes, so we optionally add one small open model (e.g., Qwen-7B) to test size dependence. Temperature 0 throughout.

Baselines. Direct and CoT are the baseline modes (mirroring [2]); within each mode, the canonical serialization is the baseline for each perturbation axis (mirroring [3]); **Graph-as-Code is the transcription-free control for Code-1shot.**

Metrics. (i) Mean accuracy per (task, mode, axis); (ii) *consistency*: exact-match agreement across the K variants (normalized output spread for numeric tasks [3]); (iii) the *invariance gap* $\Delta = \text{consistency}(\text{mode}) - \text{consistency}(\text{Direct})$ per axis — the headline quantity; (iv) code-failure-type distribution vs. perturbation type and serialization length. *Success criterion*: a statistically supported verdict among H1–H3 per task and axis, with the failure taxonomy explaining the mechanism; per course policy, a rigorous negative or mixed result is a full result.

Assumptions and requirements. (a) *No training, no GPUs*: ~ 6 tasks $\times$ 50 graphs $\times$ K variants $\times$ 4 modes (a few thousand calls per model) (Graph-as-Code adds ~ 3 –5 short calls per instance), adjustable by sampling; funded by free academic credits or a modest shared budget. (b) A sandboxed Python execution environment, which we implement ourselves. (c) All data and the perturbation taxonomy are public. (d) *First literature-review step*: a close reading of Finkelshtein et al. [2] and Cai et al. [4] to state precisely which robustness measurements they contain, sharpening our contribution’s boundary.

References

- [1] B. Fatemi, J. Halcrow, B. Perozzi. Talk like a Graph: Encoding Graphs for Large Language Models. *ICLR 2024*. arXiv:2310.04560. (*course paper list*, §10)
- [2] B. Finkelshtein, S. Cucerzan, S. K. Jauhar, R. W. White. Actions Speak Louder than Prompts: A Large-Scale Study of LLMs for Graph Inference. *ICLR 2026*. arXiv:2509.18487. (*course paper list*, §10)
- [3] D. Herbst, L. Karbeska, D. Kumar, A. Ahuja, F. Gholamzadeh Nasrabadi, F. Frasca. Lost in Serialization: Invariance and Generalization of LLM Graph Reasoners. arXiv:2511.10234. AAAI 2026 GCLR Workshop.
- [4] Q. Cai, Z. Wang, S. Diao, J. Kwok, Y. Song. CodeGraph: Enhancing Graph Reasoning of LLMs with Code. *AAAI 2025*. arXiv:2408.13863.
- [5] X. Guo, A. Li, Y. Wang, S. Jegelka, Y. Wang. G1: Teaching LLMs to Reason on Graphs with Reinforcement Learning (Erdős benchmark). *NeurIPS 2025*. arXiv:2505.18499.
- [6] J. Roh, V. Gandhi, S. Anilkumar, A. Garg. Chain-of-Code Collapse: Reasoning Failures in LLMs via Adversarial Prompting in Code Generation. arXiv:2506.06971.
- [7] L. Sarker, M. Downing, A. Desai, T. Bultan. Assessing, Exploiting, and Mitigating Syntactic Robustness Failures in LLM-Based Code Generation. *FORGE 2026*. arXiv:2404.01535.